

Task 'Unit 7 Secure Coding Practices in Object-Oriented Programming' Unit 7

Task Collaborative Discussion 3

The security vulnerabilities in the code are as follows: passwords are stored in plaintext, authentication depends on is being compared directly to a string and there is no validation or rate limiting. Plaintext storage discloses credentials if breached, while weak username and passwords combination and unlimited login attempts increase the risk of successful brute-force attacks.

```
```python
```

```
import bcrypt, re, time
```

```
class Auth:
```

```
 def __init__(self):
```

```
 self.users = {}
```

```
 self.attempts = {}
```

```
 self.lockout_time = {}
```

```
 def valid_pw(self, pw):
```

```
 return len(pw) >= 8 and re.search(r"[A-Z]", pw) and re.search(r"\d", pw)
```

```
 def add_user(self, u, p):
```

```
 if not u.strip() or not self.valid_pw(p):
```

## Task 'Unit 7 Secure Coding Practices in Object-Oriented Programming' Unit 7

```
raise ValueError("Invalid credentials")
```

```
self.users[u] = bcrypt.hashpw(p.encode(), bcrypt.gensalt())
```

```
def authenticate(self, u, p):
```

```
 if u in self.lockout_time:
```

```
 if time.time() - self.lockout_time[u] < 30:
```

```
 return False
```

```
 else:
```

```
 self.attempts[u] = 0
```

```
 del self.lockout_time[u]
```

```
 stored = self.users.get(u)
```

```
 if not stored:
```

```
 ok = False
```

```
 else:
```

```
 ok = bcrypt.checkpw(p.encode(), stored)
```

```
 if ok:
```

```
 self.attempts[u] = 0
```

## Task 'Unit 7 Secure Coding Practices in Object-Oriented Programming' Unit 7

```
else:
```

```
 self.attempts[u] = self.attempts.get(u, 0) + 1
```

```
 if self.attempts[u] >= 5:
```

```
 self.lockout_time[u] = time.time()
```

```
 return bool(ok)
```

```
auth = Auth()
```

```
auth.add_user("admin", "Pass9Secure1")
```

```
print(auth.authenticate("admin", "Pass9Secure1"))
```

```
...
```

The refactored version improves security by concealing passwords, validating inputs, enforcing stronger passwords, and limiting repeated failed logins attempts. bcrypt prevents plaintext exposure because only the password hashes are stored, not the passwords itself in plaintext, while checkpw() securely compares credentials, basic validation prevents the use of weak passwords what reduces typical attack vectors. Rate limiting reduces the risk of successful brute-force attempts by restricting repeated failure attempts. These measures are suggested by the OWASP secure password storage recommendation.

## **Task 'Unit 7 Secure Coding Practices in Object-Oriented Programming' Unit 7**

The hashing algorithm Argon2id is considered as a better alternative and best practice, due to the stronger features, nevertheless, bcrypt was chosen due to its simplicity.

(OWASP Foundation, 2026).

### **References**

- OWASP Foundation (2026) Password Storage Cheat Sheet. Available at: [https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.md](https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Password_Storage_Cheat_Sheet.md) [Accessed: 27 May 2026].

### **Use of Digital Tools**

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.